

GRAPH ALIGNMENT IS ALL YOU NEED

Marc Lelarge

INRIA, DI/ENS, PSL Research University

Machine Learning and Signal Processing on Graphs - September 2026

Learning to solve a combinatorial problem

Chaining GNNs to refine an alignment and initialize a classical solver.

Marc Lelarge, *Chaining 2-FWL GNNs for Combinatorial Graph Alignment*.

<https://arxiv.org/abs/2510.03086>

<https://github.com/mllelarge/chaining-gnn-graph-alignment>

Learning structural representations

Using alignment to benchmark GNNs and learn positional encodings.

Joint work with Adrien Lagesse, *Graph Alignment for Benchmarking Graph Neural Networks and Learning Positional Encodings*.

<https://arxiv.org/abs/2505.13087>

<https://github.com/adrien-lagesse/graph-alignment-benchmark>

Combinatorial Optimization and Reasoning with Graph Neural Networks

Quentin Cappart

*Department of Computer Engineering and Software Engineering
Polytechnique Montréal
Montréal, Canada*

QUENTIN.CAPPART@POLYMTL.CA

Didier Chételat

*CERC in Data Science for Real-Time Decision-Making
Polytechnique Montréal
Montréal, Canada*

DIDIER.CHETELAT@POLYMTL.CA

Elias B. Khalil

*Department of Mechanical & Industrial Engineering
University of Toronto
Toronto, Canada*

KHALIL@MIE.UTORONTO.CA

Andrea Lodi

*Jacobs Technion-Cornell Institute
Cornell Tech and Technion - IIT
New York, USA*

ANDREA.LODI@CORNELL.EDU

Christopher Morris

*Department of Computer Science
RWTH Aachen University
Aachen, Germany*

MORRIS@CS.RWTH-AACHEN.DE

Petar Veličković

*DeepMind
London, UK*

PETARV@DEEPMIND.COM

Cracking nuts with a sledgehammer: when modern graph neural networks do worse than classical greedy algorithms

Maria Chiara Angelini^{1,2}

Federico Ricci-Tersenghi^{1,2,3}

Cracking nuts with a sledgehammer: when modern graph neural networks do worse than classical greedy algorithms

Maria Chiara Angelini^{1,2}

Federico Ricci-Tersenghi^{1,2,3}

Despite careful attempts, Böther et al. (2022) were incapable of reproducing the results, even reporting that using random weights in the GNN yields similar results as the trained weights. Thus, at this moment this approach should be considered at best inconclusive (...)

from Cappart et al. (2023)

Graph Alignment Problem (GAP)

Given two $n \times n$ adjacency matrices A and B , the graph alignment problem is to minimize $\|A - PBP^T\|_F$ over all permutation matrices P and where $\|\cdot\|_F$ is the Frobenius norm:

$$\text{GAP} = \min_{\pi \in S_n} \sum_{i,j} (A_{ij} - B_{\pi(i)\pi(j)})^2,$$

where π is the permutation associated to the permutation matrix P . We denote by $\pi^{A \rightarrow B}$ a solution to the graph alignment problem.

Graph Alignment Problem (GAP)

Given two $n \times n$ adjacency matrices A and B , the graph alignment problem is to minimize $\|A - PBP^T\|_F$ over all permutation matrices P and where $\|\cdot\|_F$ is the Frobenius norm:

$$\text{GAP} = \min_{\pi \in S_n} \sum_{i,j} (A_{ij} - B_{\pi(i)\pi(j)})^2,$$

where π is the permutation associated to the permutation matrix P . We denote by $\pi^{A \rightarrow B}$ a solution to the graph alignment problem.

For unweighted graphs, the coefficients of the matrices A and B are in $\{0, 1\}$, hence $\pi^{A \rightarrow B}$ also solves:

$$\max_{\pi \in S_n} \sum_{i,j} A_{ij} B_{\pi(i)\pi(j)},$$

which is finding a maximum common subgraph in G_A and G_B , known to be APX-hard.

For a candidate permutation π , we report:

- the accuracy defined by

$$\mathbf{acc}(\pi, \pi^{A \rightarrow B}) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(\pi(i) = \pi^{A \rightarrow B}(i)). \quad (1)$$

- the number of common edges defined by

$$\mathbf{nce}(\pi) = \frac{1}{2} \sum_{i,j} A_{ij} B_{\pi(i)\pi(j)} \leq \mathbf{nce}(\pi^{A \rightarrow B}). \quad (2)$$

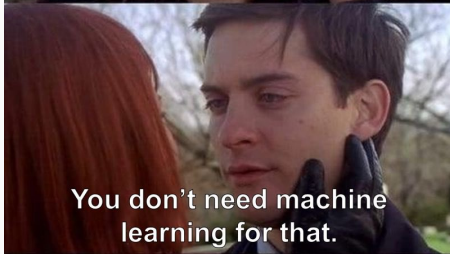
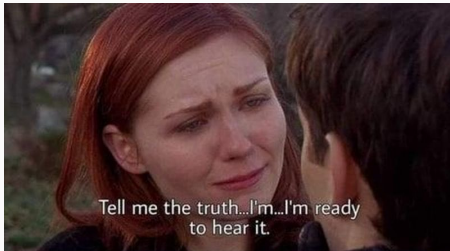
Random pairs of graphs $(\mathbf{G}_A, \mathbf{G}_B)$ such that the marginals are the same, i.e. the laws of $\mathbf{G}_A$ and $\mathbf{G}_B$ are identical but $\mathbf{G}_A$ and $\mathbf{G}_B$ are correlated. This correlation allows us to control the difficulty of the graph alignment problem. Then a random permutation $\pi^* \in \mathcal{S}_n$ is applied on the nodes of $\mathbf{G}_B$ to get $\mathbf{G}'_B$ and the training is done on the generated triplets $(\mathbf{G}_A, \mathbf{G}'_B, \pi^*)$.

Random pairs of graphs $(\mathbf{G}_A, \mathbf{G}_B)$ such that the marginals are the same, i.e. the laws of $\mathbf{G}_A$ and $\mathbf{G}_B$ are identical but $\mathbf{G}_A$ and $\mathbf{G}_B$ are correlated. This correlation allows us to control the difficulty of the graph alignment problem. Then a random permutation $\pi^* \in \mathcal{S}_n$ is applied on the nodes of $\mathbf{G}_B$ to get $\mathbf{G}'_B$ and the training is done on the generated triplets $(\mathbf{G}_A, \mathbf{G}'_B, \pi^*)$.

3 parameters:

the **number of nodes** n , the **average degree** d and the **noise level** p_{noise} .

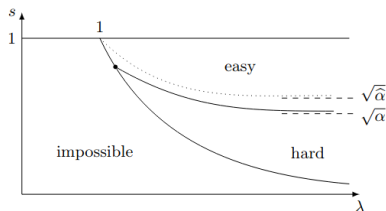
On average, $\mathbf{G}_A$ and $\mathbf{G}_B$ have $nd/2 = \mathbb{E}[\sum_{ij} A_{ij}/2]$ edges and the noise level p_{noise} controls the number of edges that are different between $\mathbf{G}_A$ and $\mathbf{G}_B$ so that the average number of common edges is $(1 - p_{\text{noise}})nd/2 = \mathbb{E}[\sum_{ij} A_{ij}B_{ij}/2]$.



Recovering the planted permutation (without learning)

Faster algorithms for the alignment of sparse correlated Erdős-Rényi random graphs

Andrea Muratori¹ and Guilhem Semerjian²



Otter's threshold: $\sqrt{\alpha} \approx 0.581$.

Ganassali et al. (2021b), Ganassali et al. (2021a), Piccioli et al. (2022), Ding et al. (2021), Mao et al. (2023), Muratori and Semerjian (2024)

Replace permutations by doubly stochastic matrices $D \in \mathcal{D}_n$.

- **Convex relaxation** supplies an initialization:

$$D_{\text{cx}} \in \arg \min_{D \in \mathcal{D}_n} \|AD - DB\|_F^2.$$

- **FAQ** uses Frank–Wolfe on the indefinite objective:

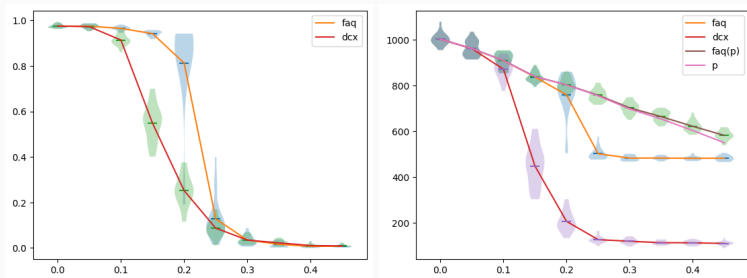
$$\max_{D \in \mathcal{D}_n} \langle AD, DB \rangle.$$

A linear assignment problem projects the result onto permutations:

$$\mathbf{Proj}(D) \in \arg \max_{P \in \mathcal{S}_n} \langle P, D \rangle.$$

Initialization matters: $\mathbf{FAQ}(D_{\text{cx}})$ is a stronger baseline than simply projecting D_{cx} , or starting FAQ from a uniform matrix.

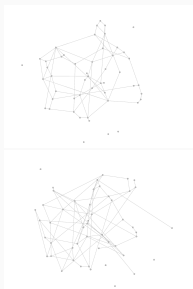
Classical baselines: Erdős-Rényi ($n = 500, d = 4$)



Accuracy (left) and number of common edges (right) as a function of noise.

Learning with graph symmetries

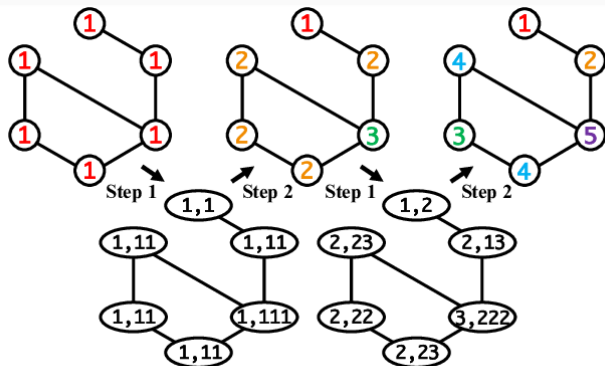
Learning the graph alignment problem with Siamese GNNs



$$\begin{array}{l} G_1 \in \{0, 1\}^{n^2} \xrightarrow{\text{GNN}} E_1 \in \mathbb{R}^{n \times b} \\ G_2 \in \{0, 1\}^{n^2} \xrightarrow{\text{GNN}} E_2 \in \mathbb{R}^{n \times b} \end{array} \quad \begin{array}{c} \nearrow \\ \nwarrow \end{array} \quad E_1 E_2^T \in \mathbb{R}^{n^2}$$

- The same GNN is used for both graphs.
- From the node similarity matrix $E_1 E_2^T$, we extract a mapping from nodes of G_1 to nodes of G_2 (using **Proj** to get a permutation).

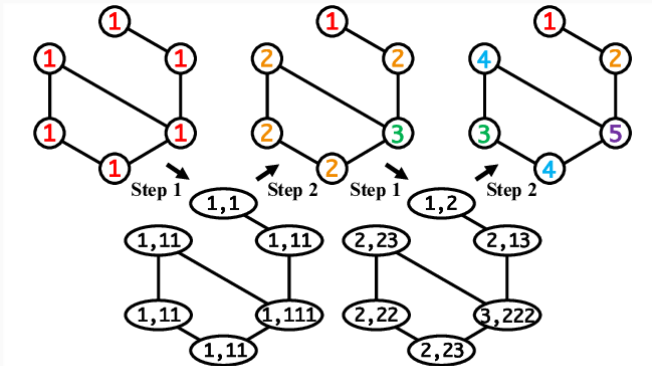
MPNN and Weisfeiler-Lehman



Step 1: generate signature strings. Step 2: sort signature strings and recolor.

Xu et al. (2019): MPNNs are at most as powerful as 1-WL. GIN reaches this bound under suitable assumptions.

MPNN and Weisfeiler-Lehman



Step 1: generate signature strings. Step 2: sort signature strings and recolor.

Xu et al. (2019): MPNNs are at most as powerful as 1-WL. GIN reaches this bound under suitable assumptions.

$$\min_{D \in \mathcal{D}_n} \|AD - DB\|_F^2 = 0 \text{ iff } G_A \text{ and } G_B \text{ cannot be distinguished by 1-WL.}$$

For FGNNs Maron et al. (2019), **messages are associated with pairs of vertices** as opposed to MPNN where messages are associated with vertices.

A residual version of this layer

$$h_{i \rightarrow j}^{t+1} = h_{i \rightarrow j}^t + f_1 \left(h_{i \rightarrow j}^t, \sum_{\ell} h_{i \rightarrow \ell}^t \odot f_0(h_{\ell \rightarrow j}^t) \right),$$

where $f_0 : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $f_1 : \mathbb{R}^{2d} \rightarrow \mathbb{R}^d$ are multilayer perceptrons (MLP) combined with a final graph normalization layer and where $\odot$ is the component-wise multiplication.

For FGNNs Maron et al. (2019), **messages are associated with pairs of vertices** as opposed to MPNN where messages are associated with vertices.

A residual version of this layer

$$h_{i \rightarrow j}^{t+1} = h_{i \rightarrow j}^t + f_1 \left(h_{i \rightarrow j}^t, \sum_{\ell} h_{i \rightarrow \ell}^t \odot f_0(h_{\ell \rightarrow j}^t) \right),$$

where $f_0 : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $f_1 : \mathbb{R}^{2d} \rightarrow \mathbb{R}^d$ are multilayer perceptrons (MLP) combined with a final graph normalization layer and where $\odot$ is the component-wise multiplication.

Expressivity theorem: (without graph normalization) a $2K$ -layer FGNN with $4d$ channels simulates any K -layer PGNN with d channels, thus reaches 2-FWL expressivity.

Decode the similarity matrix by linear assignment:

$$\pi = \mathbf{Proj}(S), \quad \pi \in \arg \max_{\sigma \in S_n} \sum_i S_{i, \sigma(i)}.$$

From a permutation to node features

Decode the similarity matrix by linear assignment:

$$\pi = \mathbf{Proj}(S), \quad \pi \in \arg \max_{\sigma \in S_n} \sum_i S_{i, \sigma(i)}.$$

Score each node by the number of edges preserved by the current alignment:

$$s(i) = \sum_j A_{ij} B_{\pi(i)\pi(j)}.$$

Order the nodes so that $s(r^A(1)) \geq \dots \geq s(r^A(n))$, and transfer that order to B :

$$r^B(k) = \pi(r^A(k)).$$

The next GNN uses these paired rankings as node features.

From a permutation to node features: example

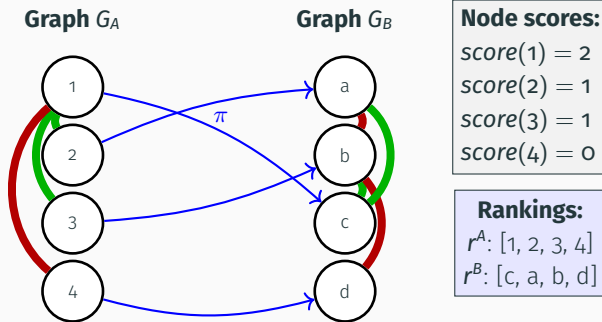


Figure 2: Illustration of Step 2. The permutation π maps $1 \rightarrow c$, $2 \rightarrow a$, $3 \rightarrow b$, $4 \rightarrow d$. Green edges show matches: edge 1-2 with a-c, and edge 1-3 with b-c. Node 1 has the highest score (2 matched edges), nodes 2 and 3 each have 1 matched edge, and node 4 has no matched edges.

Initial embeddings and similarities

$$S^{(0)} = f(A)f(B)^T.$$

Discrete feedback at each step

$$S^{(\ell-1)} \xrightarrow{\text{Proj}} \pi^{(\ell-1)} \xrightarrow{\text{edge scores}} (r^{A,(\ell-1)}, r^{B,(\ell-1)}).$$

Refinement using paired rankings

$$S^{(\ell)} = g^{(\ell)}(A, r^{A,(\ell-1)}) g^{(\ell)}(B, r^{B,(\ell-1)})^T.$$

Train each network in turn, keeping the preceding networks fixed. The ranking step is not differentiable.

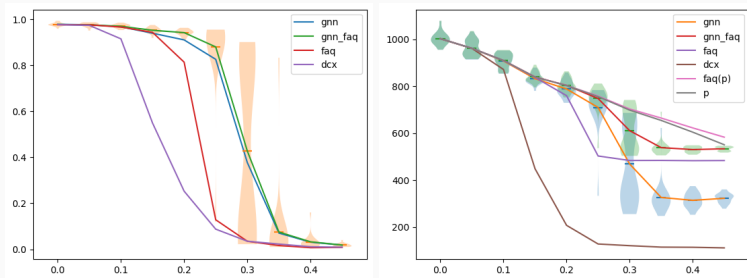
Each network minimizes cross-entropy against the planted permutation:

$$\mathcal{L}(S, \pi^*) = - \sum_i \log(\text{softmax}(S_{i,\cdot}))_{\pi^*(i)}.$$

- **Training:** fit f , freeze it, then fit $g^{(1)}$, and continue sequentially.
- **Inference:** decode with **Proj** or use the learned alignment to initialize **FAQ**.
- **Looping:** reuse the final network and keep the candidate with the largest observed **nce**.

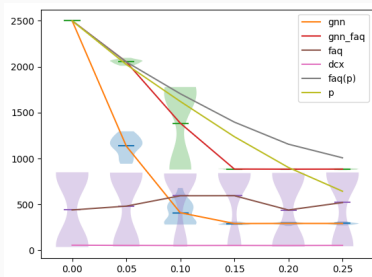
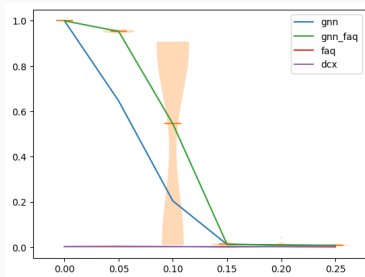
Keeping the best candidate guarantees that looping cannot reduce the returned **nce**. It gives no such guarantee for planted-node accuracy.

Chained GNNs: Erdős–Rényi ($n = 500, d = 4$)



Accuracy (left) and number of common edges (right) as a function of noise.

Regular ($n = 500, d = 10$)



Accuracy (left) and number of common edges (right) as a function of noise.

Results on noisy real networks

Table 1: Accuracy (**acc**) and number of common edges (**nce**) on noisy versions of real-world networks. Each network is corrupted by adding noise at different levels. ChFGNN ER₄ is trained on Erdős-Rényi graphs, while ChFGNN is trained on the specific network and noise level. In bold if gain in **nce** is larger than 2%.

METHOD	REAL-WORLD NETWORKS WITH ADDED NOISE (ACC / NCE)					
	YEAST25LC		CA-NETSCIENCE		INF-EUROROAD	
	5%	10%	10%	20%	10%	20%
FUGAL [2024]	53.1 / 7480	44.6 / 7035	60.3 / 794	37.7 / 629	18.3 / 818	2.9 / 714
FAQ (D_{cx})	49.8 / 7660	44.7 / 7245	65.2 / 822	45.6 / 687	55.8 / 1170	10.9 / 940
ChFGNN ER ₄	47.6 / 7693	42.3 / 7297	63.5 / 818	44.1 / 688	40.0 / 1111	7.5 / 970
ChFGNN	54.1 / 7732	51.3 / 7404	65.4 / 824	57.0 / 724	63.5 / 1213	15.4 / 963
MAX NCE	7909	7498	826	730	1272	1137

What chaining contributes

- **FAQ**(D_{cx}) is a strong baseline. Learned initializations improve it on hard correlated instances.
- On sparse Erdős–Rényi graphs ($n = 500$, $d = 4$, $p_{\text{noise}} = 0.25$), ChFGNN-**FAQ** reaches **85% accuracy**, compared with **13%** for **FAQ**(D_{cx}).
- Pair representations help on regular graphs, where constant-feature MPNNs cannot distinguish vertices.
- Dataset-specific training improves real-network alignment. Transfer from synthetic graphs is more mixed.

Cost: each FGNN layer uses $O(n^2 d)$ activation memory and $O(n^3 d)$ arithmetic for pair aggregation.

Lelarge, Sections 5–6.

Benchmarking GNNs and learning positional encodings

Joint work with Adrien Lagesse

Graph Alignment for Benchmarking Graph Neural Networks and Learning Positional Encodings

The same Siamese architecture can compare different GNNs:

$$X = f_{\theta}(G), \quad \tilde{X} = f_{\theta}(\tilde{G}), \quad S = X\tilde{X}^T.$$

- Constant node features and no input edge features isolate topology.
- Noise controls task difficulty within each graph family.
- Learned node embeddings can be reused for downstream tasks.

Alignment datasets from existing graphs

1. Take a graph from a base dataset and remove its input features.
2. Perturb its edges, then apply a random permutation π^* .
3. Train the Siamese GNN to recover this planted correspondence.

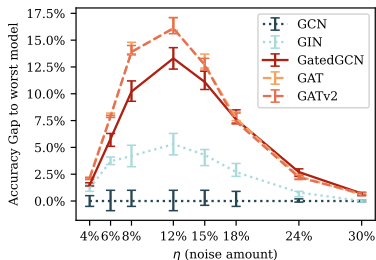
Base graphs	Preparation	Edge noise
Erdős-Rényi	Synthetic	Add and remove
AQSOL, PCQM4Mv2	Molecular graphs	Add only
CoraFull, OGBN-Arxiv	BFS subgraphs	Add and remove

Five graph families, eight noise levels from 4% to 30%: **40 alignment datasets**.

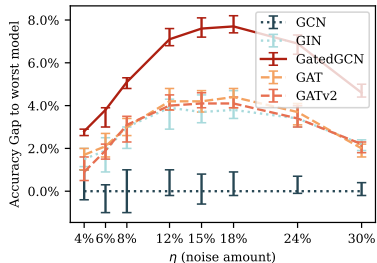
Addition-only noise preserves all original edges, so the planted alignment is optimal but may be non-unique. With edge deletion, optimality is not guaranteed for arbitrary graphs.

Task difficulty separates architectures

Erdős-Rényi



CoraFull



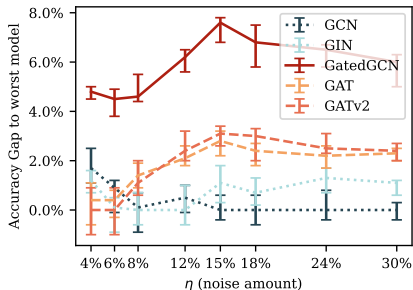
Vertical axis: accuracy gap relative to the worst model at the same noise level.

Intermediate noise gives the largest separation. Very easy or very difficult instances compress the differences.

Lagesse and Lelarge, Figure 3. Five MPNNs, 10 runs per setting.

Different performance within the 1-WL bound

AQSOL



Isotropic models

GCN, GIN

Anisotropic models

GatedGCN, GAT, GATv2

Anisotropic models perform better in these experiments, even with constant input features.

All five models use four layers, approximately 60 000 parameters and 64-dimensional output embeddings.

The comparison measures learned structural representations within the 1-WL expressivity bound.

Lagesse and Lelarge, Section 5, Figure 3 and model specifications.

Pre-train on alignment

$$X = f_{\theta}(G) \in \mathbb{R}^{n \times d}.$$

The main positional-encoding comparison uses $d = 32$:

Pre-training graphs	Encoder	Noise η
AQSOL	GatedGCN	15%
ZINC	GatedGCN	30%
PCQM4Mv2	GAT	30%

Pre-train on alignment

$$X = f_{\theta}(G) \in \mathbb{R}^{n \times d}.$$

The main positional-encoding comparison uses $d = 32$:

Pre-training graphs	Encoder	Noise η
AQSOL	GatedGCN	15%
ZINC	GatedGCN	30%
PCQM4Mv2	GAT	30%

Reuse the node embeddings in a transformer

$$z_i = \text{AtomEmbedding}(a_i) + \text{GAPE}(G)_i.$$

The transformer only sees atom types plus the positional encoding: no 3D coordinates and no bond features, so all structural information must come from the encoding.

Validation MAE, mean $\pm$ standard deviation over four runs. Lower is better.

Encoding	AQSOL	PCQM4Mv2	ZINC
None	1.71 $\pm$ 0.05	0.236 $\pm$ 0.004	0.658 $\pm$ 0.009
Laplacian	1.31 $\pm$ 0.006	0.155 $\pm$ 0.003	0.171 $\pm$ 0.008
ABS-LAP	1.27 $\pm$ 0.009	0.162 $\pm$ 0.004	0.208 $\pm$ 0.012
R-LAP	1.23 $\pm$ 0.007	0.191 $\pm$ 0.008	0.500 $\pm$ 0.042
RWPE	1.12 $\pm$ 0.004	0.158 $\pm$ 0.005	0.169 $\pm$ 0.003
SignNet	1.26 $\pm$ 0.006	0.137 $\pm$ 0.003	0.116 $\pm$ 0.005
GPSE	1.105 $\pm$ 0.008	0.135 $\pm$ 0.005	0.104 $\pm$ 0.006
GAPE	1.085 $\pm$ 0.009	0.133 $\pm$ 0.003	0.147 $\pm$ 0.008
GAPE + RWPE	1.069 $\pm$ 0.009	0.125 $\pm$ 0.004	0.109 $\pm$ 0.005

GAPE + RWPE has the lowest reported mean on AQSOL and PCQM4Mv2. GPSE has the lowest mean on ZINC.

Reconstructing edges from frozen GAPE

Freeze the alignment-trained GatedGCN. Train a transformer decoder to reconstruct the adjacency matrix using only its node embeddings.

Target	Pre-training graphs	Accuracy	F1
ZINC	ZINC	98.8%	94.7%
ZINC	CoraFull	98.7%	94.0%
ZINC	PCQM4Mv2	98.6%	94.6%
ZINC	OGBN-Arxiv	98.7%	94.4%
PCQM4Mv2	ZINC	97.7%	94.5%
PCQM4Mv2	CoraFull	97.7%	94.0%
PCQM4Mv2	PCQM4Mv2	97.8%	94.5%
PCQM4Mv2	OGBN-Arxiv	97.8%	94.3%

The embeddings retain substantial structural information, including across pre-training domains. These are **edge-level** scores, not exact whole-graph recovery rates.

What the benchmark tells us

- Alignment tests structural representation without task-specific node or edge labels.
- A noise sweep reveals where architectures differ most. The useful noise level depends on graph topology.
- Alignment pre-training supplies positional encodings that support molecular regression and edge reconstruction.

Scope: these results concern the evaluated graph families and architectures. They do not establish a universal ranking for downstream tasks or remove the 1-WL bound of the MPNN encoders.

Code and dataset generation:

github.com/adrien-lagesse/graph-alignment-benchmark

For combinatorial optimization

Chaining turns alignment predictions into structural feedback. Expressive pair representations give useful initializations for **FAQ**.

For evaluating and pre-training GNNs

Corrupted graph pairs provide supervision and adjustable difficulty. The resulting embeddings transfer to positional encoding and reconstruction tasks.

Next: is 1-WL enough?

Chaining relied on 2-FWL pair representations, while GAPE relied on 1-WL message passing. The last part asks when a graph learning task actually requires the extra expressivity.

Joint work with Adrien Lagesse

What does 1-WL see? Start from node features. At each round, update each node's color using its own color and the multiset of its neighbors' colors (and edge features, when present).

The stable color multiset is the graph's **1-WL fingerprint**, denoted by $\text{CR}(G)$.

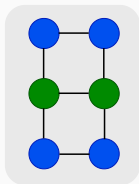
An amenable graph has a unique fingerprint, up to relabeling. Every graph with the same fingerprint must be a copy of G :

$$\text{CR}(H) = \text{CR}(G) \implies H \cong G.$$

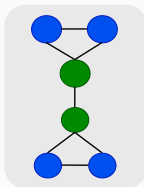
Here $H \cong G$ means that only the node names differ.

This must hold for **every graph** H , including graphs absent from the dataset. Checking only the observed graphs is weaker.

A pair of graphs that 1-WL cannot distinguish



G



H

In both graphs:

- each blue node has one blue and one green neighbor;
- each green node has two blue and one green neighbor.

1-WL keeps the same two color classes at every subsequent round.

Both fingerprints contain **four blue and two green nodes**, but H has triangles and G has none. They are non-isomorphic.

H is a **confuser** of G : neither graph is amenable. Even if a dataset contains only G , the confuser still exists.

Amenability on real molecular datasets

Dataset	Split	Random GNN separation	1-WL separation	Amenability
ZINC-250k	Train	99.37%	100.00%	99.37%
	Validation	99.91%	100.00%	99.42%
	Test	99.98%	100.00%	99.48%
AQSO	Train	91.64%	100.00%	94.60%
	Validation	92.48%	100.00%	85.77%
	Test	97.90%	100.00%	71.77%

Structural statistics. Separation is measured within each reported split.

- **ZINC:** about 99.4% of graphs are amenable. A randomly initialized GNN already separates over 99%.
- **AQSO:** 1-WL distinguishes all graphs within each split. On the test split, 71.77% are additionally **certified amenable**: their fingerprint identifies them even among graphs outside the dataset.

One task, three uses. Corrupted graph pairs give supervision with adjustable difficulty, without node labels, edge labels or task-specific annotation.

1. **Solve it.** Chained 2-FWL GNNs refine coarse alignments into initializations that **FAQ** turns into accurate matchings on hard sparse and regular instances.
2. **Benchmark with it.** At a calibrated noise level, alignment ranks 1-WL architectures on structure alone. Anisotropic models perform better, even with constant input features.
3. **Pre-train with it.** Alignment-trained node embeddings serve as positional encodings for transformers and reconstruct the adjacency matrix from frozen features.

Expressivity is a property of the data. Amenability tells us when 1-WL already identifies a graph: almost every ZINC graph is certified amenable, far fewer AQSOL graphs are. Measure it before paying for a more expressive model.

Thank you!

- M. Böther, O. Kißig, M. Taraz, S. Cohen, K. Seidel, and T. Friedrich. What's wrong with deep learning in tree search for combinatorial optimization. *ICLR*, 2022.
- Q. Cappart, D. Chételat, E. B. Khalil, A. Lodi, C. Morris, and P. Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.
- J. Ding, Z. Ma, Y. Wu, and J. Xu. Efficient random graph matching via degree profiles. *Probability Theory and Related Fields*, 179:29–115, 2021.
- L. Ganassali, L. Massoulié, and M. Lelarge. Correlation detection in trees for planted graph alignment. *arXiv preprint arXiv:2107.07623*, 2021a.
- L. Ganassali, L. Massoulié, and M. Lelarge. Impossibility of partial recovery in the graph alignment problem. In *Conference on Learning Theory*, pages 2080–2102. PMLR, 2021b.
- A. Lagesse and M. Lelarge. Graph alignment for benchmarking graph neural networks and learning positional encodings. <https://arxiv.org/abs/2505.13087>, 2026a.
- A. Lagesse and M. Lelarge. Does your graph learning task really require more expressive GNNs? in preparation, 2026b.
- M. Lelarge. Chaining 2-FWL GNNs for combinatorial graph alignment. <https://arxiv.org/abs/2510.03086>, 2026.
- C. Mao, Y. Wu, J. Xu, and S. H. Yu. Random graph matching at otter's threshold via counting chandeliers. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1345–1356, 2023.

- H. Maron, H. Ben-Hamu, H. Serviansky, and Y. Lipman. Provably powerful graph networks. *Advances in neural information processing systems*, 32, 2019.
- A. Muratori and G. Semerjian. Faster algorithms for the alignment of sparse correlated erdős-rényi random graphs, 2024.
- G. Piccioli, G. Semerjian, G. Sicuro, and L. Zdeborová. Aligning random graphs with a sub-tree similarity message-passing algorithm. *Journal of Statistical Mechanics: Theory and Experiment*, 2022(6):063401, 2022.
- K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=ryGs6iA5Km>.